

```
"Ulf", SqlByteString "Norell"]]
```

You can also run an update query to the database. For example:

```
ghci> run conn "UPDATE names set lname = 'Peyton Jones' WHERE
fname = 'Simon'" []
1
ghci> commit conn
```

Verifying the output from the `Sqlite3` command prompt, you get:

```
sqlite> select * from names;
1|Edwin|Brady
2|Simon|Peyton Jones
3|Ulf|Norell
```

The `HDBC` driver provides many functions to retrieve information about the database and the drivers. A few examples are illustrated below:

```
ghci> hdbcDriverName conn
"sqlite3"
ghci> hdbcClientVer conn
"3.8.4.3"
ghci> dbTransactionSupport conn
True
ghci> getTables conn
["names"]
ghci> describeTable conn "names"
[("id", SqlColDesc {colType = SqlIntegerT, colSize = Nothing,
colOctetLength = Nothing, colDecDigits = Nothing, colNullable
= Nothing}), ("fname", SqlColDesc {colType = SqlUnknownT
"varchar(80)", colSize = Nothing, colOctetLength = Nothing,
colDecDigits = Nothing, colNullable = Nothing}), ("lname",
SqlColDesc {colType = SqlUnknownT "varchar(80)", colSize =
Nothing, colOctetLength = Nothing, colDecDigits = Nothing,
colNullable = Nothing})]
```

It is considered good practice to use `handleSqlError` before running any `HDBC` commands to catch errors that may arise during the database transactions. For example:

```
ghci> handleSqlError $ quickQuery conn "SELECT * from namesaaa"
[]
*** Exception: user error (SQL error: SqlError {seState = "",
seNativeError = 1, seErrorMsg = "prepare 23: SELECT * from
namesaaa: no such table: namesaaa"})
```

To disconnect from the database, you can use the `disconnect` function provided by `HDBC` as shown below:

```
ghci> disconnect conn
```

Let's now look at how to access a MySQL database using

the `mysql-simple` package. You will also need to create or grant privileges for an existing user to use the MySQL database server. Please follow your GNU/Linux distribution manual on how to install and configure a MySQL server. On Fedora, for example, you must have a `mysql` and `mysql-server` installed:

```
$ sudo yum install community-mysql community-mysql-server
```

You can install the `mysql-simple` Haskell package using the following command:

```
$ cabal install mysql-simple
```

Create a *test* database using the `mysql` command line tool as shown below:

```
$ mysql -u user -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.5.38-log MySQL Community Server (GPL)
Copyright (c) 2000, 2014, Oracle and/or its affiliates.
All rights reserved. Oracle is a registered trademark of
Oracle Corporation and/or its affiliates. Other names may be
trademarks of their respective owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current
input statement.
mysql> create database test;
Query OK, 1 row affected (0.03 sec)
```

Let us create a users' table that has an ID as well as first name and last name fields, using the `mysql-simple` package, as follows:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.MySQL.Simple
main :: IO ()
main = do
  conn <- connect defaultConnectInfo
  { connectUser = "user"
  , connectPassword = "password"
  , connectDatabase = "test"
  }
  execute conn "create table users (id INT, fname
VARCHAR(80), lname VARCHAR(80))" ()
  close conn
```

The `OverloadedStrings` extension allows string literals to be polymorphic for the `IsString` class. The `defaultConnectInfo` is of type `ConnectInfo`:

```
ghci> :t defaultConnectInfo
defaultConnectInfo :: ConnectInfo
```

`ConnectInfo` can take many parameters to describe the